Blake Brandon

AI Systems Engineer | Backend Python Developer (LLMs/RAG)

blakebrandon.dev@gmail.com | github.com/blakebrandon-hub | linkedin.com/in/blake-e-brandon
Portfolio: blakebrandon-hub.github.io

SUMMARY

Self-taught engineer with no formal CS background — started at 30, line by line, before agentic tools existed. Specializes in AI systems and backend Python, building reliable, real-time, stateful applications with LLM orchestration and data pipelines. Experienced in enforcing structure on non-deterministic outputs through closed-loop pipelines (generation → validation → repair), RAG via embeddings and client-side vector memory, provider-agnostic AI integration (Gemini/Claude/GPT), and resolving concurrency in collaborative systems. Track record spans client delivery, internal tooling, and a public platform with an active user base. Focused on determinism, data integrity, and production-scale systems.

TECHNICAL SKILLS

Languages: Python, JavaScript, SQL
Backend: Flask, Django, REST APIs, WebSockets, SQLAlchemy
AI/LLMs: OpenAI, Claude, Gemini, RAG, embeddings, client-side vector memory, multi-provider orchestration, structured prompting
Data: PostgreSQL, Supabase
Systems: AWS, Vercel, Docker, concurrency, schedulers, transactional systems

SELECTED PROJECTS

Unified Developer Interface (UDI) — *Multi-Agent Dev Workbench*

- Built a Flask-based developer workbench unifying multi-provider LLM chat, a sandboxed terminal/file manager, and a configurable multi-agent pipeline in one browser interface
- Designed a multi-agent pipeline engine supporting executor + critic agent chains that iteratively refine output until validation passes or a max-iteration limit is reached
- Implemented a structured tag protocol (*[CODE], [RUN], [LINT], [TEST], [QUERY]*, etc.) enabling agents to write files, execute shell commands, and run tests — sandboxed to a restricted working directory with blocked dangerous commands and path-traversal protection
- Built interactive human-in-the-loop pausing, allowing a running pipeline to halt and request clarification mid-execution before resuming
- Added provider-agnostic LLM routing (Gemini/Claude/GPT) with automatic provider inference from model name, plus retry logic with exponential backoff and hard timeouts

Signals: *multi-agent orchestration, sandboxed execution, structured LLM control protocols, human-in-the-loop systems, multi-provider abstraction*

RAG Vector Store — *Semantic Memory Search Engine*

- Built a Flask-based vector store demonstrating semantic search and memory management for RAG pipelines, using SBERT sentence embeddings to encode text into high-dimensional vectors
- Implemented cosine-similarity search over stored embeddings, filtering results by a similarity threshold and returning the top-k most relevant matches
- Designed memory pruning strategies: age-based expiration, size-based capping, and duplicate detection/removal via similarity threshold
- Built an interactive dashboard for managing stored memories, monitoring model load status, and inspecting real-time search scores

Signals: *RAG, embeddings, vector search, semantic memory management*

Ulite Lighting — *E-Commerce Platform (Client Project)*

- Built and deployed a production e-commerce platform for a commercial client ($1,200 contract)

- Designed product catalog (100+ SKUs) with filtering, technical documentation, and structured data management
- Integrated Supabase backend for inventory, assets, and content delivery

Signals: *production systems, client delivery, full-stack ownership*

CRM Platform *— Offline-First Data & Automation System*

- Built offline-first ingestion system with conflict-aware sync (local → server)
- Designed automated email campaign pipeline with deterministic state updates
- Improved engagement by 35% while maintaining data consistency

Signals: *offline-first systems, data integrity, reliable pipelines*

AI RPG Engine *— Shared Narrative Framework (powers 9 titles on AI Playable Fiction)*

- Built a reusable Flask/LLM narrator engine (*app.py*) reskinned across 9 genre-distinct text RPGs — dark fantasy, cyberpunk, horror, sci-fi survival, monster-collecting — each with unique mechanics and art direction on one core backend
- Designed a stateless narration architecture: backend receives system prompt, context, history, and player action, and returns consequence-driven prose that never editorializes and punishes vague input
- Built fully provider-agnostic AI integration with independent routing for narration and image generation — e.g., Claude can narrate while Imagen or GPT-Image renders scene art — with automatic provider detection from model name
- Implemented The Stone, a client-side vector memory (Transformers.js) embedding every exchange and surfacing semantically relevant past events as LLM context — reused across all titles
- Built persistent JSON-serialized game state per title: stats, inventory/weight systems, dual-tier currencies, faction tracking, quest/vow systems, and genre-specific resource mechanics (spell-cost “Fracture,” cyberware “Neural Strain”)

Signals: *reusable system architecture, stateful LLM applications, cross-provider AI orchestration, client-side vector memory/RAG, prompt-constrained narrative generation*

AI Playable Fiction *— Multi-Genre AI Game Platform & Community*

- Built a public platform for discovering and publishing AI-narrated games — browse/filter by genre, developer devlogs, user accounts, and a dual-path game submission system supporting either direct external links or GitHub-repo-to-Vercel deployment with environment variable configuration
- Grew the platform to 21 published titles (9 self-authored) across genres including horror, fantasy, sci-fi, and mystery, with an active companion community of 200+ members on r/AIPlayableFiction
- Designed the submission/publishing pipeline handling repo verification, deploy orchestration, and per-game environment variable injection (e.g., API keys) for third-party developer submissions

Signals: *full-stack platform development, developer tooling, deployment orchestration, community building, product ownership*